

```
$ pytest -v
```

СОВРЕМЕННЫЕ ТЕХНОЛОГИИ ТЕСТИРОВАНИЯ ПО

pytest

Фреймворк модульного тестирования для языка Python

Группа: _____

Участники: _____

Что такое pytest

НАЗВАНИЕ

pytest
(раньше — py.test)

АВТОР

Хольгер Крекель
(Holger Krekel)

ГОД СОЗДАНИЯ

2004 — py.test
2010 — pytest 2.0

ОРГАНИЗАЦИЯ

Сообщество pytest-dev,
лицензия MIT

ЯЗЫК

Python 3
(CPython и PyPy)

ПЛАТФОРМЫ

Windows, Linux, macOS

Стандарт тестирования в Python

№ 1

Популярность

Самый используемый фреймворк тестирования по опросам Python Developers Survey (PSF и JetBrains)

1000+

Сообщество

Плагинов в официальном каталоге, сотни контрибьюторов на GitHub, подробная документация

9.x

Обновления

Несколько релизов в год: 8.0 — 2024, 9.0 — 2025; поддержка новых версий Python

Что умеет pytest

Ассерты

Обычный `assert`; при ошибке `pytest` показывает, какие значения не совпали

Фикстуры

`@pytest.fixture`: подготовка данных, области видимости, общий `conftest.py`

Параметризация

`@pytest.mark.parametrize` — один тест на много наборов данных

Моки

`monkeypatch`, `unittest.mock`, плагин `pytest-mock` — подмена зависимостей

Отчёты

`-v`, `--junitxml`, `pytest-html`, покрытие кода через `pytest-cov`

CI/CD и IDE

GitHub Actions, GitLab CI, Jenkins; PyCharm, VS Code; `tox`, `nox`

Сильные и слабые стороны

+ Плюсы

- Короткий синтаксис: функции и `assert`
- Мощные фикстуры и параметризация
- 1000+ плагинов на любые задачи
- Запускает и старые тесты `unittest`
- Бесплатный, открытый, стандарт индустрии

– Минусы

- Работает только с Python
- «Магия» фикстур непонятна новичкам
- Не входит в стандартную библиотеку
- Большие наборы тестов медленные без `xdist`
- `conftest.py` скрывает зависимости

От Hello, world к фикстурам

Простейший тест · test_hello.py

```
def hello(name):  
    return f"Hello, {name}!"  
  
def test_hello():  
    assert hello("world") == "Hello, world!"
```

Вывод результатов

```
$ pytest -v  
test_hello.py::test_hello PASSED  
test_shop.py::test_price[apple-50] PASSED  
test_shop.py::test_price[milk-90] PASSED  
===== 3 passed in 0.02s =====
```

Сложнее: фикстура + параметры · test_shop.py

```
import pytest  
  
@pytest.fixture  
def cart():  
    return {"apple": 50, "milk": 90}  
  
@pytest.mark.parametrize("item, price", [  
    ("apple", 50),  
    ("milk", 90),  
)  
def test_price(cart, item, price):  
    assert cart[item] == price
```

pytest, unittest и nose2

Критерий	pytest	unittest	nose2
Поставка	Отдельный пакет (pip)	Стандартная библиотека	Отдельный пакет
Синтаксис	Функции + assert	Классы TestCase, self.assertEqual	Как unittest
Фикстуры	Гибкие, с областями видимости	setUp / tearDown	setUp / tearDown
Параметризация	Встроенная	subTest	Через плагин
Плагины	1000+	Нет	Немного
Развитие	Активное	Вместе с Python	Слабое

Также: **Hypothesis** — property-based тесты, работает поверх pytest; **Robot Framework** — приёмочные тесты

Запуск за четыре шага

- 1 `python -m venv .venv`
- 2 `pip install pytest`
- 3 `pytest --version`
- 4 Создать `tests/test_*.py` и выполнить **pytest**

pytest сам находит файлы `test_*.py` и функции `test_*`

Минимальная конфигурация · `pyproject.toml`

```
[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = "test_*.py"
addopts = "-v"
```

Где используется pytest

Тесты известных открытых проектов написаны на pytest:

pandas

NumPy

scikit-learn

FastAPI

Flask

requests

pip

Веб-разработка

Тесты API и сервисов на Django (pytest-django), Flask, FastAPI

UI-автотесты

Microsoft развивает официальный плагин pytest-playwright; Selenium

Data Science и ML

Проверка расчётов, моделей и конвейеров обработки данных

Куда движется pytest



Новые версии Python

Поддержка появляется вместе с релизами языка



Асинхронный код

pytest-asyncio и anyio для тестов async/await



Типизация

Аннотации типов и проверка через mypy



ИИ-ассистенты

Нейросети генерируют тесты именно в формате pytest

Долгосрочно: 20+ лет развития и статус стандарта — pytest останется актуальным

pytest — выбор по умолчанию для Python

Когда использовать

- Любой проект на Python — от учебного до крупного
- Нужны фикстуры и много наборов данных
- Тесты запускаются в CI/CD

Рекомендации

- Начинать с простых `assert`
- Общие фикстуры — в `conftest.py`
- Подключить `pytest-cov` и `pytest-xdist`
- В CI запускать с `--junitxml`



Вопросы?

Спасибо за внимание!

===== all tests passed =====